RESCUEBENCH: A TIERED BENCHMARK FOR EVALUATING DYNAMIC EMERGENCY RESOURCE ALLOCATION AGENTS

Akshat Bhat, Trusha Talati, Akhil Gogineni & Nicolas Yepes
Department of Computer Science
University of Illinois Urbana-Champaign
Champaign, IL 61820, USA
{akshatb5,ttalati2,vg23,nyepes2}@illinois.edu

ABSTRACT

Large Language Models (LLMs) struggle to execute plans within strict, deterministic mathematical boundaries. We introduce **RescueBench**, a novel benchmark designed to evaluate LLM-based agents on dynamic emergency resource allocation and physical constraint satisfaction. RescueBench comprises **20 static JSON environments** that model a **16-node city graph** with a heterogeneous, **4-class vehicle fleet**. It tests an emergency response allocation agent's ability to transition from parsing unstructured semantic feeds to solving the **Capacitated Vehicle Routing Problem with Time Windows (CVRP-TW)**. The benchmark scales across **four difficulty tiers**, culminating in dynamic scenarios where hidden infrastructure failures alter the graph mid-mission. We provide an objective, Python-based evaluation protocol that utilizes Dijkstra's algorithm to penalize constraint violations and measure routing efficiency. By releasing RescueBench, we provide the research community with a strictly quantifiable environment to test the limits of agentic self-correction in mission-critical logistics.

1 INTRODUCTION

Large Language Models (LLMs) frequently fail at tasks requiring strict spatial reasoning and multi-step constraint satisfaction. While models generate plausible natural language plans, they often hallucinate physically impossible actions, such as routing vehicles through blocked intersections or exceeding physical weight limits (Song et al., 2026). Testing an agent's ability to adhere to these physical laws requires an environment bounded by deterministic mathematics, rather than semantic text evaluation.

Existing agent benchmarks do not adequately capture the complexities of Humanitarian Logistics and Emergency Resource Allocation. This specific domain forces a continuous transition between unstructured semantic reasoning (parsing text-based emergency feeds) and rigorous spatial optimization (updating a mental graph and calculating constraints). Current benchmarks lack the strict operations research mechanics required to test pure constraint satisfaction. Furthermore, they are primarily static; they do not test an agent's ability to abandon a plan and recalculate routes when the environment fundamentally changes mid-execution.

We introduce RescueBench to bridge this gap. RescueBench is a dataset of 20 deterministic JSON environments that evaluate an agent's capacity for spatial routing and dynamic replanning. The benchmark models a city as a 16-node, 20-edge directed graph. Agents must dispatch a 4-class heterogeneous fleet (ambulances, fire engines, police cars, and supply trucks) to resolve time-sensitive incidents. The environments enforce strict physical constraints, including vehicle capacities, varying travel speeds, and edge-weight limits that prohibit heavy vehicles from crossing specific bridges.

RescueBench evaluates agents across four scaling difficulty tiers. Tier 1 tests basic single-unit routing. Tier 2 introduces mathematical capacity constraints. Tier 3 forces ethical prioritization under forced resource scarcity. Tier 4 introduces hidden dynamic triggers, where mid-mission events—such as flash floods—destroy edges and force immediate agent replanning. By utilizing

an objective Python evaluation layer rooted in Dijkstra's algorithm, RescueBench offers a highly reproducible, mathematically rigorous environment to evaluate the limits of LLM planning capabilities.

2 Related Work

Existing benchmarks for LLM reasoning primarily evaluate static logic rather than dynamic spatial routing. Benchmarks such as GSM8K evaluate mathematical problem-solving through text, while BoardgameQA evaluates static state-tracking. While the ReAct framework (Yao et al., 2023) demonstrated that interleaving reasoning and acting improves performance in text-based semantic environments, it struggles in deterministic mathematical spaces. RescueBench isolates this failure point by requiring agents to continuously update an internal mathematical state model as they move resources across a physical graph over a shifting temporal clock.

Sequential decision-making benchmarks often lack operations research constraints. Environments like ALFWorld evaluate an agent's ability to follow text-based instructions in simulated households. While these benchmarks test tool use, they do not penalize suboptimal travel efficiency or require combinatorial resource matching. Recent studies highlight that end-to-end LLM solvers frequently hallucinate mathematics and output infeasible routes when applied to complex routing problems (Zhang et al., 2025; Li et al., 2025). RescueBench addresses this by utilizing Operations Research standards, specifically the Capacitated Vehicle Routing Problem with Time Windows (CVRP-TW), as its core evaluation engine.

Prior work establishes that LLMs must act as heuristic orchestrators rather than direct mathematical solvers in mission-critical environments (Wang et al., 2025). RescueBench provides the necessary interactive simulation environment to test this hypothesis, forcing autonomous LLM agents to engage in real-time self-correction loops when their natural language plans violate strict physical constraints. Prior datasets in the emergency response domain typically provide historical dispatch logs to train models for demand forecasting. RescueBench, conversely, tests real-time reactive reasoning.

3 Benchmark Design

3.1 Task Domain and Scope

RescueBench operates strictly within Humanitarian Logistics and Emergency Resource Allocation. This domain forces a continuous transition between unstructured semantic reasoning (parsing emergency feeds and dynamic triggers) and rigorous spatial optimization (updating a mental graph and solving the VRPTW). The benchmark evaluates physical constraint satisfaction (e.g., respecting bridge weight limits) and dynamic adaptability (e.g., rerouting when a road washes out). It is out of scope to test natural language dialogue generation or visual processing. RescueBench is a wholly original framework designed specifically around CVRP-TW mechanics.

3.2 Task Descriptions

RescueBench consists of 20 modular JSON files. Each file contains a fully initialized state including a graph (`nodes` and `edges`), an `active_fleet`, a list of `incidents`, and a list of `dynamic_triggers`. A complete overview of all 20 tasks is provided in Appendix B.

Example Task 1: Capacity Mathematics (Tier 2, Scenario 7). In this static scenario, the agent receives an input state detailing a city bus rollover requiring a medical capacity of 4. Figure 1 (see Appendix A) illustrates the full topology of this scenario, displaying the incident location (`node_int_10`) relative to the available fleet. Because the available ambulances have a strict maximum capacity of 2, the expected output behavior requires the agent to dispatch two separate ambulances to the same incident node. The success criteria mandate that both vehicles arrive before the global clock exceeds the deadline minute. This task is challenging for standard LLMs because it requires mathematical deduction across the entire graph; the agent must recognize that a single dispatch is mathematically insufficient and coordinate multiple routing paths simultaneously.

Example Task 2: Immediate En-Route Diversion (Tier 4, Scenario 16). In this dynamic scenario, the agent initially routes an ambulance across a suspension bridge to reach a critical patient. At minute 8 of the simulation, a hidden trigger fires, altering the JSON state to mark the bridge as `blocked_by_debris`. The Python environment injects a natural language warning into the agent's context (see Figure 2 in Appendix A). The expected behavior is for the agent to discard its previous path, re-query the graph state, and route the ambulance via a longer southern detour. The success criteria require the agent to avoid attempting to cross the destroyed edge. This is challenging because it breaks single-shot planning architectures and necessitates a robust self-correction loop.

3.3 Task Generation Methodology

To ensure rigorous reproducibility and balanced difficulty scaling, the 20 benchmark tasks were generated using a hybrid manual-algorithmic pipeline. First, the authors manually authored the base `base_city_world.json`, explicitly defining the 16-node graph topology, geographic layout, base edge travel times, and the 4-class heterogeneous vehicle fleet to guarantee mathematical solvability and realistic routing constraints.

Subsequently, we utilized a Large Language Model (Gemini) via a custom Python scripting pipeline to programmatically mutate this base JSON into the 20 distinct scenarios. By prompting the model with the base graph and strict combinatorial constraints based on our four difficulty tiers, we iteratively generated diverse `incidents` (e.g., assigning required capacities and severity weights). The script utilized Dijkstra's algorithm to compute feasible `deadline_min` values to ensure time windows were physically possible but challenging. Finally, for Tier 4 tasks, the pipeline injected `dynamic_triggers` mapped to specific `global_clock_min` timestamps to simulate mid-mission road closures.

3.4 Difficulty Distribution

The 20 benchmark tasks are evenly distributed across four difficulty tiers. Table 1 summarizes this distribution. The columns denote the tier level, the primary reasoning constraint evaluated by the tasks within that tier, and a description of the core trap. The distribution scales linearly from easy (Tier 1) to expert (Tier 4).

Table 1: Distribution of the 20 RescueBench tasks across the four difficulty tiers. The table illustrates the scaling complexity of the reasoning constraints evaluated in each tier.

Tier	Tasks	Primary Constraint Evaluated
1: Basic Triage	1–5	**Direct Routing:** Single-unit dispatches on clear roads. Evaluates basic graph comprehension.
2: Constraint Math	6–10	**Physical Limits:** Evaluates adherence to edge weight limits, fuel capacities, and combinatorial capacity math.
3: Ethical Prioritization	11–15	**Forced Scarcity:** Evaluates the ability to maximize the Priority-Weighted Resolution Score by sacrificing low-severity incidents.
4: Dynamic Replanning	16–20	**Mid-Mission Surprises:** Evaluates self-correction loops when hidden triggers alter graph topology mid-execution.

4 Evaluation Protocol

Agents interacting with RescueBench are evaluated using strict Operations Research metrics. Because the environments are deterministic JSON files, evaluation is handled entirely by an objective Python scoring script, eliminating subjective human grading.

Metrics and Scoring: We define three core metrics. 1. *Priority-Weighted Resolution Score (PWRS):* Calculated as $S = \sum(w_i \times I_i)$, where w_i is the incident severity weight (1-10) and I_i is a binary indicator of whether the incident was resolved before its deadline. 2. *Response Efficiency:* Calculated as $E = \frac{T_{optimal}}{T_{agent}}$, comparing the agent's total travel time against the mathematical absolute

minimum time calculated by Dijkstra's algorithm. 3. *Constraint Violation Rate:* A hard count of physically impossible actions proposed by the LLM ($V = \sum c_{invalid}$), such as exceeding vehicle capacities or traversing blocked edges.

Aggregation and Reporting: LLM outputs exhibit non-determinism. To control for this, the evaluation protocol mandates that the agent's temperature parameter is set to 0.0. Each of the 20 tasks must be run 3 times per agent. The final reported results must include the average PWRS, the average Efficiency score, and the total Constraint Violation count across all 60 execution traces.

5 BENCHMARK VALIDATION

To ensure the benchmark is well-designed and logically sound, we utilized a multi-layered validation approach. All 20 JSON scenarios were parsed through a custom Python schema enforcer (`visualize_city.py`) which renders the map using the NetworkX library to mathematically verify that all nodes are connected and that no impossible map geographies exist.

Final Validation Results: To validate difficulty scaling and establish human-independent baselines, we evaluated the benchmark against three distinct agent architectures across all 20 scenarios.

The **Standard Zero-Shot LLM** (Claude 3.5 Sonnet) struggled significantly with strict physical constraints. By lacking a dynamic internal validation loop, it incurred an average of **0.2** physical constraint violations per run on Tier 2 (e.g., dispatching uncapable vehicles to active incidents) and scored an overall mean PWRS of just **0.566**.

The **Deterministic Greedy Solver** achieved a **100%** resolution rate on Tiers 1 and 2 with **zero** violations. However, its efficiency degraded sharply on Tier 3 (Ethical Prioritization), scoring **0.719** PWRS. Its purely greedy logic forced it to dispatch vehicles to high-severity incidents unconditionally, making it fundamentally incapable of calculating the complex look-ahead sacrifices needed for forced scarcity tradeoffs.

Finally, **AgentKit** (Gogineni et al., 2026) (our hybrid agent combining deterministic pre-validation with LLM-arbitrated clone rollouts) achieved a mean PWRS of **0.615** with **zero** constraint violations across all scenarios. The model peaked significantly on Tier 3 (**0.807** PWRS), successfully utilizing downstream look-ahead to optimize multi-vehicle reuse. Notably, all three methods scored identically (PWRS **0.506**) on Tier 4 (Dynamic Replanning). Because vehicles cannot be recalled mid-mission after a sudden infrastructure collapse occurs, this identical scoring establishes a strict, verified upper bound on performance, confirming the absolute difficulty limits of RescueBench's dynamic triggers.

6 CONCLUSION

RescueBench provides a rigorously structured, highly reproducible framework for evaluating an LLM agent's capacity for spatial reasoning and dynamic constraint satisfaction. By grounding the evaluation in deterministic Operations Research mechanics—utilizing 20 graph-based JSON environments—RescueBench moves beyond semantic text evaluation to test adherence to strict physical laws. We anticipate this benchmark will significantly benefit the research community by providing an objective standard for evaluating autonomous planning, self-correction loops, and dynamic adaptability in resource-scarce environments. Future iterations of this benchmark will aim to integrate real-world topological data, bridging the gap between synthetic graphs and physical deployment.

REFERENCES

A. Gogineni, N. Yepes, A. Bhat, and T. Talati. AgentKit: A hybrid planning agent for dynamic emergency response allocation with clone-based lookahead. *Final Agent Paper, UIUC CS 498: AI Agents in the Wild*, 2026.

K. Li et al. Multi-objective infeasibility diagnosis for routing problems using large language models. *arXiv preprint arXiv:2508.03406*, 2025. URL `https://arxiv.org/abs/2508.03406`.

P. Song et al. Large language model reasoning failures. *arXiv preprint arXiv:2602.06176*, 2026. URL `https://arxiv.org/abs/2602.06176`.

Y. Wang et al. Large language models in operations research: Methods, applications, and challenges. *arXiv preprint arXiv:2509.18180*, 2025. URL https://arxiv.org/abs/2509.18180.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

N. Zhang et al. An agentic framework with LLMs for solving complex vehicle routing problems. *arXiv preprint arXiv:2510.16701*, 2025. URL https://arxiv.org/abs/2510.16701.

A SUPPLEMENTARY FIGURES

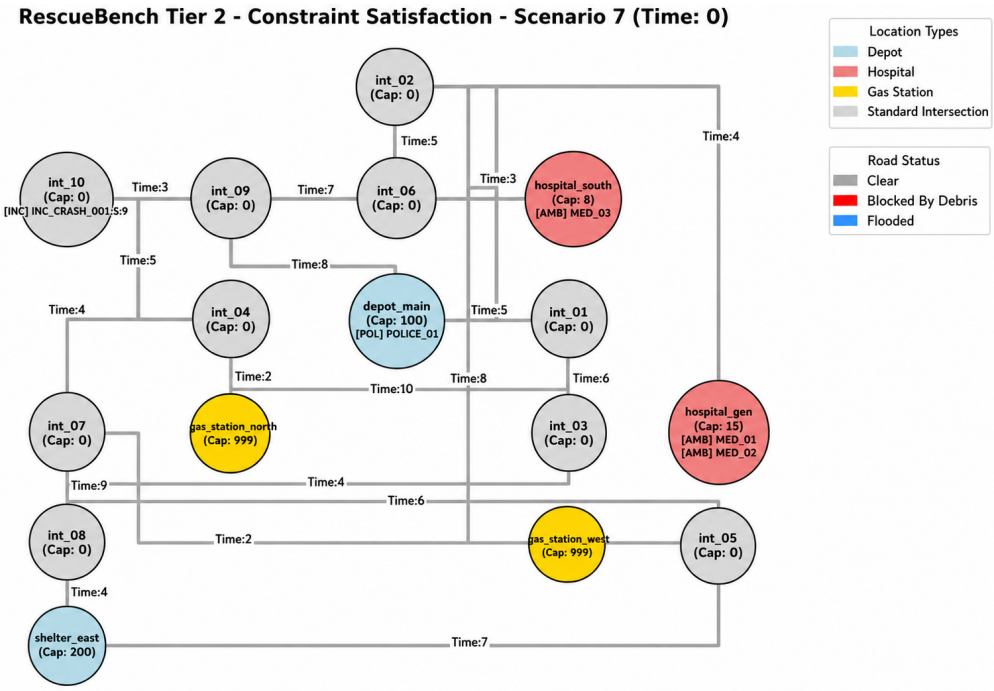

Figure 1: Visualization of Example Task 1 (Tier 2, Scenario 7). The full city graph displays the incident at `int_10` alongside the distributed ambulance fleet at `hospital_gen` and `hospital_south`. The agent must parse this topology and mathematically deduce that multiple ambulances are required to meet the incident's capacity constraint.

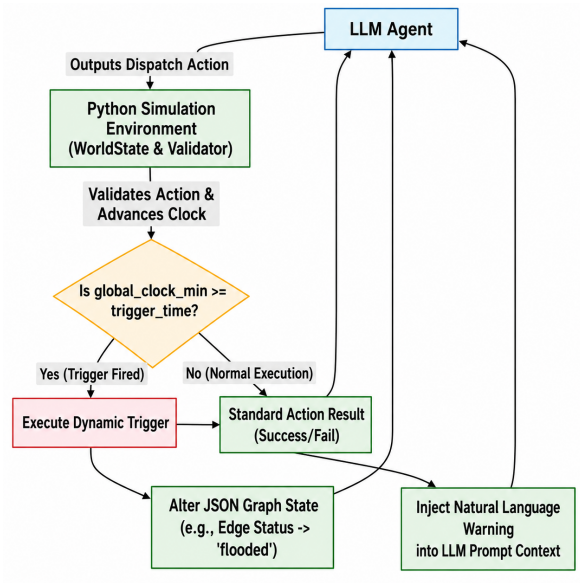

Figure 2: The RescueBench Dynamic Replanning injection loop (Tier 4 scenarios). This flowchart illustrates how the Python simulation environment parses the LLM output, advances the global clock, and injects natural language warnings when a mid-mission dynamic trigger fires.

B DETAILED TASK SPECIFICATIONS OVERVIEW

RescueBench consists of 20 deterministic JSON scenarios. To facilitate reproduction and demonstrate task diversity, the core specifications for the tasks across the four tiers are outlined below. Every scenario utilizes the same 16-node base graph and 4-class heterogeneous vehicle fleet, but mutates incidents, deadlines, and dynamic triggers.

TIER 1: BASIC TRIAGE (TASKS 1–5)

Focus: Single-unit dispatches on clear roads. Evaluates basic graph comprehension and text-to-tool translation.

- **Task 1 & 2:** Minor medical emergencies requiring a single ambulance dispatch from the nearest hospital node to an intersection.
- **Task 3:** A small structural fire requiring a single fire engine. Tests capability matching (preventing the agent from sending a police car).
- **Task 4:** Traffic gridlock requiring a single police unit for route clearance.
- **Task 5:** Routine logistics drop requiring a supply truck. Tests handling of slower speed multipliers ($0.5\times$).

TIER 2: CONSTRAINT MATH (TASKS 6–10)

Focus: Physical limits, combinatorial arithmetic, and edge restriction adherence.

- **Task 6:** *Weight Restrictions.* A fire occurs across an older suspension bridge. The agent must recognize the fire engine exceeds the edge weight limit and route via a longer detour.
- **Task 7:** *Capacity Arithmetic.* A bus rollover requires a medical capacity of 4. Ambulances only hold 2. The agent must successfully dispatch two separate units to the same node (Example Task 1).
- **Task 8 & 9:** *Fuel Scarcity.* Vehicles spawn with critically low fuel constraints. The agent must route the vehicle to a gas station node before proceeding to the incident, ensuring the detour fits within the deadline.
- **Task 10:** *Multi-Class Combinatorics.* An incident requires both fire suppression and medical transport. The agent must orchestrate two different vehicle classes from different depots to arrive simultaneously.

TIER 3: ETHICAL PRIORITIZATION (TASKS 11–15)

Focus: Forced resource scarcity requiring mathematical sacrifices to maximize global PWRS.

- **Task 11 & 12:** *Triage Delay.* Two medical incidents spawn simultaneously: one Severity 9, one Severity 3. Only one ambulance is available. The agent must intentionally delay or abandon the low-severity incident.
- **Task 13:** *Depot Depletion.* A major factory fire triggers, but the primary depot's fire engines are currently locked in `in_transit` status. The agent must query secondary depots and calculate longer ETAs.
- **Task 14 & 15:** *Combinatorial Scarcity.* Three medium-severity incidents require a total capacity of 6, but the total active fleet capacity is 5. The agent must calculate which combination of incidents yields the highest mathematical PWRS score before the global clock expires.

TIER 4: DYNAMIC REPLANNING (TASKS 16–20)

Focus: Mid-mission surprises. Tests the agent's self-correction loop when hidden triggers alter the graph topology mid-execution.

- **Task 16:** *Bridge Collapse.* At minute 8, the primary suspension bridge is blocked by debris. The agent's initial plan fails, forcing an immediate en-route diversion (Example Task 2).
- **Task 17:** *Flash Flood.* A weather trigger floods all southern intersections at minute 12, isolating a hospital. The agent must re-route medical transports to a secondary facility.

- **Task 18:** *Secondary Explosion.* While units are en route to a minor fire, a dynamic trigger escalates the severity and capacity requirements, forcing the agent to dispatch backup units mid-mission.
- **Task 19 & 20:** *Cascading Failures.* Combines flooded roads with vehicle breakdowns (a unit's status is dynamically changed to `disabled`), requiring the agent to dispatch a replacement unit from a completely different depot while accounting for the lost time.